

MODULE 1 – AUTOMATION WITH SELENIUM

EXPERIMENT 02 – SYNCHRONIZATION AND EXPLICIT WAITS

1. Module Name and Experiment Title

Module Name: Module 1 – Automation with Selenium

Experiment Number: 02

Experiment Title: Synchronization and Explicit Waits

Programming Language: Python

Automation Tool: Selenium WebDriver

Browser: Google Chrome

Website: The Internet – Herokuapp

Website URL: https://the-internet.herokuapp.com/dynamic_loading/1

2. Problem Statement

Automate a web page containing dynamic content using Selenium WebDriver. The experiment requires navigating to a web page where a button reveals hidden text after a delay.

Implement **WebDriverWait** along with **expected_conditions** to wait until the required text element becomes visible before extracting its text.

The automation must not use `time.sleep()`. Instead, it should use explicit waits to synchronize test execution with the dynamic behavior of the web page.

3. Objective

The main objectives of this experiment are:

1. To understand the importance of synchronization in Selenium WebDriver.
2. To understand the concept of explicit waits in web automation.
3. To implement `WebDriverWait` to handle dynamically loaded web elements.
4. To use `expected_conditions` to wait until an element becomes visible.
5. To automate interaction with a button that reveals hidden content.
6. To extract text from a dynamically displayed web element.
7. To verify the expected text using assertions.

8. To understand why explicit waits are useful when handling dynamic web pages.
 9. To maximize the browser window for better visibility during the demonstration.
-

4. Tools, Software and Concepts Used

Tools and Software

- Python
- Selenium WebDriver
- Google Chrome
- ChromeDriver
- Visual Studio Code

Concepts Used

- Selenium WebDriver
 - Synchronization in Selenium
 - Explicit Waits
 - WebDriverWait
 - `expected_conditions`
 - `visibility_of_element_located()`
 - `find_element()`
 - `By.ID`
 - `By.CSS_SELECTOR`
 - `click()`
 - `text`
 - `assert` statements
 - Browser window maximization
 - Dynamic web elements
 - Exception handling
-

5. Source Code / Implementation Steps

Implementation Steps

1. Import the required Selenium libraries and modules.
2. Initialize the Chrome WebDriver.
3. Maximize the browser window.
4. Open the dynamic loading practice web page.
5. Locate the **Start** button using the appropriate locator.
6. Click the Start button to trigger the dynamic loading process.
7. Initialize `WebDriverWait` with a maximum waiting time of 15 seconds.
8. Use `expected_conditions.visibility_of_element_located()` to wait until the required text element becomes visible.
9. Once the element is visible, extract its text using the `.text` property.

10. Verify that the extracted text matches the expected output using an assertion.
11. Display the extracted text and successful execution message in the console.
12. Close the browser after completing the automation.

Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# Initialize Chrome WebDriver
driver = webdriver.Chrome()

try:
    # Maximize browser window
    driver.maximize_window()

    # Open the dynamic loading page
    driver.get(
        "https://the-internet.herokuapp.com/dynamic_loading/1"
    )

    # Locate the Start button
    start_button = driver.find_element(
        By.CSS_SELECTOR, "#start button"
    )

    # Click the Start button
    start_button.click()

    # Wait until the text becomes visible
    wait = WebDriverWait(driver, 15)

    result = wait.until(
        EC.visibility_of_element_located(
            (By.CSS_SELECTOR, "#finish h4")
        )
    )

    # Extract the displayed text
    actual_text = result.text

    # Verify the expected text
    assert actual_text == "Hello World!"

    # Display the result
    print("Text displayed:", actual_text)
    print("Synchronization successful!")

finally:
    # Close the browser
    driver.quit()
```

Explanation of Important Code Components

1. WebDriverWait

`WebDriverWait` waits for a specified condition to be satisfied, up to a maximum timeout.

In this experiment, the maximum waiting time is 15 seconds. The script proceeds as soon as the required condition is satisfied.

2. `expected_conditions`

The `expected_conditions` module provides conditions that can be used with explicit waits.

In this experiment, `visibility_of_element_located()` waits until the specified text element is visible on the web page.

3. `visibility_of_element_located()`

This condition checks whether the specified element is present in the DOM and visible on the web page.

The automation proceeds as soon as the condition is satisfied.

4. `assert`

The `assert` statement verifies that the extracted text matches the expected output.

If the text is different, the assertion fails, indicating that the expected result was not obtained.

5. `finally`

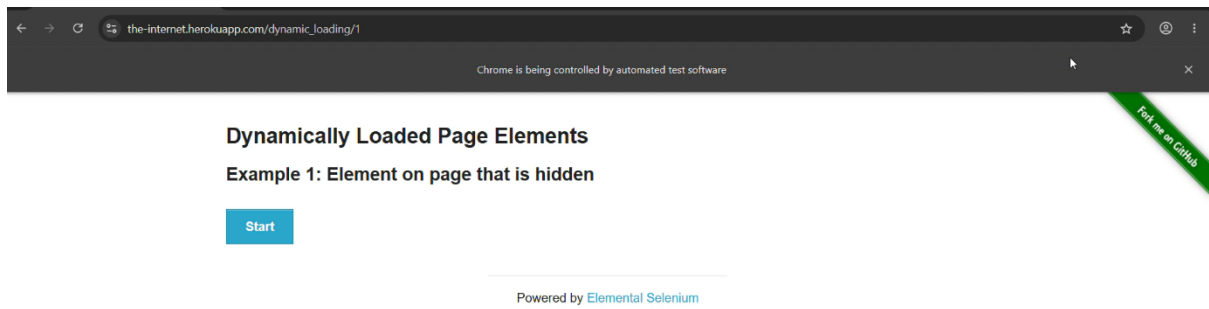
The `finally` block ensures that the browser is closed after the automation finishes, including when an exception occurs.

6. Output with Clearly Labelled Screenshots

The Selenium automation script was executed successfully on the dynamic loading practice page. The following screenshots demonstrate the initial page, dynamic content loading, successful text display, and final console output.

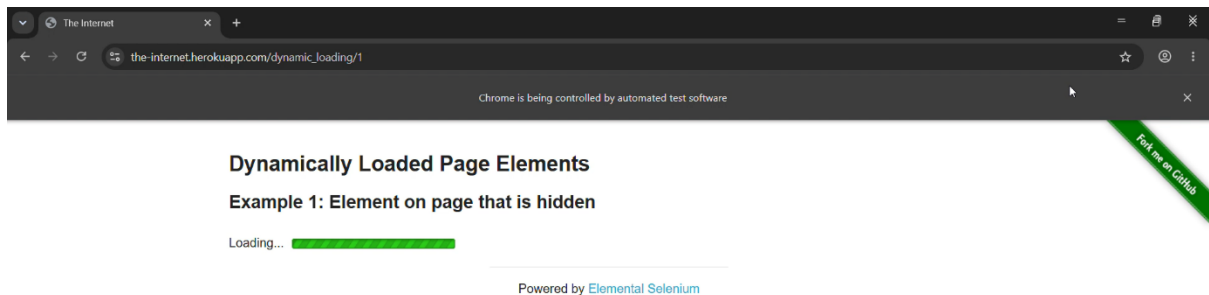
Screenshot 1: Dynamic Loading Web Page

Description: The dynamic loading practice page is opened in the maximized Chrome browser. The page displays a Start button that triggers the dynamic loading process



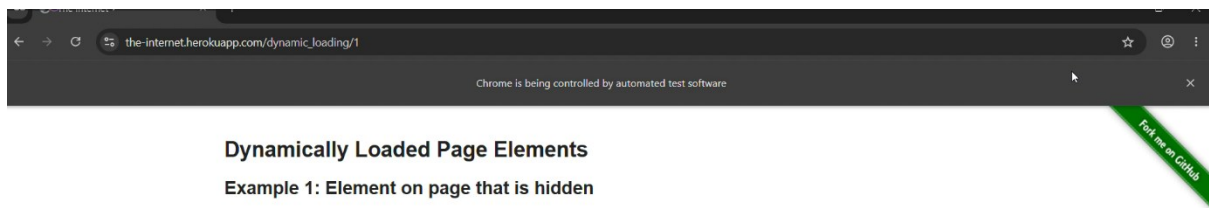
Screenshot 2: Dynamic Content Loading

Description: After clicking the Start button, the web page begins loading the hidden content. Selenium waits for the required text element to become visible using `WebDriverWait` and `expected_conditions`.



Screenshot 3: Successfully Displayed Dynamic Text

Description: After the dynamic loading process is completed, the text **Hello World!** becomes visible on the web page. Selenium identifies the element and extracts its text successfully.

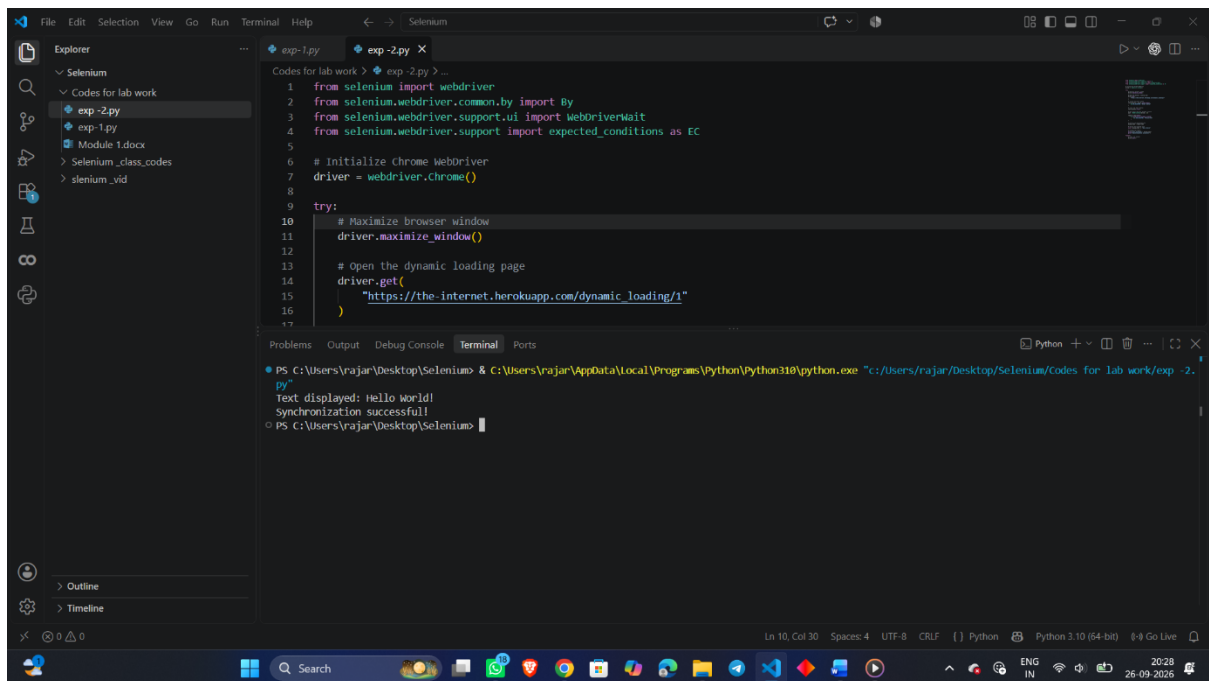


Screenshot 4: Successful Execution and Console Output

Description: The VS Code terminal displays the extracted text and the synchronization success message after the explicit wait condition is satisfied and the assertion passes.

Console Output:

Text displayed: Hello World!
Synchronization successful!



7. Result, Observation and Conclusion

Result

The Selenium WebDriver script successfully automated the dynamic loading web page using explicit waits.

The script clicked the Start button, waited until the required text element became visible, extracted the displayed text, and verified the expected output using an assertion.

The experiment was completed without using `time.sleep()`.

Observation

- Dynamic web pages may load elements after a certain period.
- Explicit waits help synchronize automation with the application's behavior.
- `WebDriverWait` allows Selenium to wait for a specified condition for a maximum duration.
- `visibility_of_element_located()` waits until the required element becomes visible.
- The automation successfully extracted the text **Hello World!** after it appeared.
- Explicit waits avoid unnecessary fixed delays and allow the script to proceed as soon as the required condition is met.

Conclusion

The experiment successfully demonstrated the use of synchronization and explicit waits in Selenium WebDriver.

It provided practical experience in handling dynamically loaded web elements using `WebDriverWait` and `expected_conditions`.

The experiment also demonstrated how explicit waits can make Selenium automation more efficient and reliable by waiting for specific conditions rather than relying on fixed delays.